O que são funções no MySQL?

Funções são recursos prontos do banco de dados que recebem um ou mais valores e retornam um resultado.

Elas podem transformar texto, calcular valores, trabalhar com datas, avaliar condições ou resumir registros.

```
SELECT UPPER('mysql') AS texto_maiusculo,
```

Resultado: MYSQL. A função UPPER recebe uma string e devolve outra string em letras maiúsculas.

Categoria	O que faz	Exemplos
Agregação	Resume valores de várias linhas	SUM, AVG, COUNT, MAX, MIN
Numéricas	Executa ou formata cálculos	ROUND, CEIL, FLOOR, ABS, MOD
Texto	Transforma ou combina strings	CONCAT, UPPER, LOWER, TRIM
Data e hora	Extrai ou calcula informações temporais	NOW, YEAR, MONTH, DATEDIFF
Controle e NULL	Escolhe resultados ou substitui ausências	IF, CASE, COALESCE

Importância prática: as funções ajudam a produzir relatórios consistentes, reduzir processamento repetido na aplicação, padronizar transformações e responder perguntas de negócio diretamente aos dados.

Cenário de dados para os exemplos

Usaremos uma tabela de vendas. Cada linha representa um registro de venda, quantidade e preço permitem calcular o valor bruto. Neste exemplo, desconto é um percentual entre 0 e 100.

```
CREATE TABLE vendas (
    id INT PRIMARY KEY AUTO_INCREMENT,
    vendedor VARCHAR(100),
    produto VARCHAR(100),
    categoria VARCHAR(50),
    quantidade INT,
    preco DECIMAL(10,2),
    data_venda DATE,
    desconto DECIMAL(5,2),
    observacao VARCHAR(255)
),
```

```
INSERT INTO vendas
(vendedor, produto, categoria, quantidade, preco, data_venda,
desconto, observacao)
VALUES
('Ana', 'Notebook', 'Informática', 2, 3500.00, '2026-01-10', 5.00,
NULL),
('Bruno', 'Mouse', 'Informática', 10, 80.00, '2026-01-11', 0.00,
'Venda à vista'),
('Ana', 'Teclado', 'Informática', 5, 150.00, '2026-01-12', 10.00,
NULL),
('Carlos', 'Cadeira', 'Móveis', 3, 900.00, '2026-01-13', 8.00,
'Entrega pendente'),
('Bruno', 'Mesa', 'Móveis', 2, 1200.00, '2026-01-14', 0.00, NULL),
('Carlos', 'Monitor', 'Informática', 4, 1100.00, '2026-01-15', 5.00,
NULL),
```

Valor bruto de cada registro: quantidade × preço.

```
SELECT produto, quantidade, preco,
       quantidade * preco AS valor_bruto
FROM vendas,
```

Observação de modelagem: em um sistema real, valores monetários devem usar DECIMAL. A regra de desconto precisa ser confirmada com o negócio, um desconto percentual não é o mesmo que um valor fixo.

Funções de agregação

Funções de agregação analisam um conjunto de linhas e retornam um resultado resumido. Sem GROUP BY, normalmente produzem um único resultado para todo o conjunto filtrado, com GROUP BY, produzem um resultado por grupo.

SUM(): soma

Soma os valores não nulos de uma expressão ou coluna. É usada, por exemplo, para totalizar itens, faturamento ou custos.

```
SELECT SUM(quantidade) AS total_itens
FROM vendas,
```

```
SELECT SUM(quantidade * preco) AS faturamento_bruto
FROM vendas,
```

Com os dados de exemplo, o total é 26 itens e o faturamento bruto é R$ 18.910,00.

Se desconto representar percentual, o faturamento líquido pode ser calculado assim: valor × (1 − desconto / 100).

```
SELECT ROUND(SUM(
    quantidade * preco * (1 - desconto / 100)
), 2) AS faturamento_liquido
FROM vendas,
```

AVG(): média

Calcula a média aritmética dos valores não nulos. Pode medir preço médio, quantidade média por registro ou valor médio por venda.

```
SELECT AVG(preco) AS preco_medio
FROM vendas,

SELECT AVG(quantidade) AS quantidade_media
FROM vendas,

SELECT ROUND(AVG(quantidade * preco), 2) AS ticket_medio
FROM vendas,
```

MAX() e MIN(): maior e menor valor

MAX retorna o maior valor e MIN o menor valor entre os valores considerados. São úteis para identificar extremos e acompanhar limites.

```
SELECT MAX(preco) AS maior_preco,
       MIN(preco) AS menor_preco
FROM vendas,

SELECT MAX(quantidade * preco) AS maior_venda
FROM vendas,

SELECT produto, preco
FROM vendas
WHERE preco = (SELECT MAX(preco) FROM vendas),
```

COUNT(): contagem

Forma	O que conta
COUNT(*)	Todas as linhas do resultado, inclusive quando há colunas NULL.
COUNT(coluna)	Somente linhas em que aquela coluna não é NULL.
COUNT(DISTINCT coluna)	Quantidade de valores diferentes não nulos.

```
SELECT COUNT(*) AS total_vendas,
       COUNT(observacao) AS observacoes_preenchidas,
       COUNT(DISTINCT vendedor) AS vendedores_diferentes
FROM vendas,
```

NULL e tratamento de ausência de valor

NULL representa ausência, desconhecimento ou não aplicabilidade de um valor. Não é zero, não é uma string vazia e não deve ser comparado com o operador de igualdade.

```
-- Incorreto: a comparação não testa NULL
SELECT * FROM vendas WHERE observacao = NULL,

-- Correto
SELECT * FROM vendas WHERE observacao IS NULL,
SELECT * FROM vendas WHERE observacao IS NOT NULL,
```

Em geral, as funções de agregação ignoram valores NULL. COUNT(*) conta a linha, COUNT(coluna) e AVG(coluna) consideram apenas valores não nulos da coluna.

COALESCE(): primeiro valor não nulo

COALESCE retorna o primeiro argumento que não seja NULL. É útil para exibir um valor padrão ou evitar que uma ausência se propague em uma expressão.

```
SELECT produto,
       COALESCE(observacao, 'Sem observação') AS observacao
FROM vendas,

SELECT produto,
       COALESCE(desconto, 0) AS desconto_aplicado
FROM vendas,
```

Boa prática: substitua NULL apenas quando houver uma regra clara. Em análises, ausência de informação pode ter significado diferente de zero.

GROUP BY: agrupando resultados

GROUP BY separa linhas em grupos com base em uma ou mais colunas.

As funções de agregação são então aplicadas a cada grupo.

É a base de relatórios por pessoa, categoria, período ou outra dimensão.

```
-- Faturamento por vendedor
SELECT vendedor,
       SUM(quantidade * preco) AS faturamento
FROM vendas
GROUP BY vendedor,

-- Itens vendidos por categoria
SELECT categoria, SUM(quantidade) AS itens_vendidos
FROM vendas
GROUP BY categoria,

-- Preço médio por categoria
SELECT categoria, AVG(preco) AS preco_medio
FROM vendas
GROUP BY categoria,

-- Quantidade de registros de venda por vendedor
SELECT vendedor, COUNT(*) AS total_vendas
FROM vendas
GROUP BY vendedor,
```

Também é possível agrupar por mais de uma coluna, cada combinação forma um grupo independente.

```
SELECT vendedor, categoria,
       SUM(quantidade * preco) AS faturamento
FROM vendas
GROUP BY vendedor, categoria,
```

Regra importante: cada coluna selecionada deve estar agrupada, ser usada em uma agregação ou ser compatível com a dependência funcional reconhecida pelo MySQL. Consultas ambíguas podem falhar ou retornar valores não determinísticos.